

ELBOs for the six batch-conditioning arms

Notation and shapes

Shapes are for the sweep configuration: $G = 200$ genes, $d_z = 3$, $d_v = 2$, $B = 5$ batches.

- $x_i \in \mathbb{Z}_{\geq 0}^G$ — observed counts for cell i (`adata.X`).
- $\tilde{x}_i = \log(1 + x_i) \in \mathbb{R}^G$ — what every encoder actually reads.
- $b_i \in \{0, \dots, B-1\}$ — pandas category code; $e_{b_i} \in \{0, 1\}^B$ its one-hot.
- $z_i \in \mathbb{R}^{d_z}$, $v_i \in \mathbb{R}^{d_v}$ — the two latents.
- $\text{theta} \in \mathbb{R}_{>0}^G$ — a `pyro.param`, one dispersion per gene, shared by all cells.
- $L_i = \sum_g x_{ig}$ — library size. $\epsilon = 10^{-5}$.

Training uses `Trace_ELBO()`, so every term below is a *sampled* log-density ratio evaluated at the guide’s reparameterized draws — not an analytic KL.

The shared skeleton

All six arms optimise

$$\mathcal{L}_i = \underbrace{\log p(x_i | z_i, \cdot)}_{\text{scale 1}} + \underbrace{\log p(z_i | v_i, \cdot) - \log q(z_i | \cdot)}_{\text{scale 1}} + \beta \underbrace{\left[\log p(v_i) - \log q(v_i | \cdot) \right]}_{\text{scale } \beta} \quad (1)$$

where z_i and v_i are drawn *by the guide*; the model’s sample sites for z and v are replayed, not re-drawn. `poutine.scale(scale=beta)` wraps only the v site, in both `model()` and `guide()`; the z and x sites sit at scale 1.

The pieces below are byte-identical across all six arms.

Guide latents.

$$(m_i^q, \tilde{s}_i^q) = \text{encoder_x_to_z}(\cdot), \quad s_i^q = \text{softplus}(\tilde{s}_i^q) + \epsilon, \quad q(z_i | \cdot) = \mathcal{N}(m_i^q, \text{diag}(s_i^q)^2) \quad (2)$$

$$(n_i^q, \tilde{r}_i^q) = v\text{-encoder}(\cdot), \quad r_i^q = \text{softplus}(\tilde{r}_i^q) + \epsilon, \quad q(v_i | \cdot) = \mathcal{N}(n_i^q, \text{diag}(r_i^q)^2) \quad (3)$$

Model latents.

$$(m_i^p, \tilde{s}_i^p) = \text{decoder_v_to_z}(v_i), \quad s_i^p = \text{softplus}(\tilde{s}_i^p), \quad p(v_i) = \mathcal{N}(0, I_{d_v}) \quad (4)$$

Likelihood. With $\mu_i = \text{softmax}(\text{decoder_z_to_x}(\cdot)) \in \Delta^{G-1}$,

$$\ell_i = \log(L_i \mu_i + \epsilon) - \log(\text{theta} + \epsilon), \quad \log p(x_i | \cdot) = \sum_{g=1}^G \log \text{NegativeBinomial}(x_{ig} | \text{theta}_g + \epsilon, \ell_{ig}) \quad (5)$$

parameterised by total count and logits.

Where the six arms differ

Exactly five slots. Everything outside this table is identical.

arm	encoder_x_to_z input	guide z mean	model z mean	v-encoder input	deco
model1-add	$\tilde{x}_i$	m_i^q	$m_i^p + \text{batch_shift}[b_i]$	$[z_i, \tilde{x}_i]$	
model2-add	$\tilde{x}_i$	$m_i^q + \text{batch_shift_post}[b_i]$	$m_i^p + \text{batch_shift_prior}[b_i]$	$[z_i, \tilde{x}_i]$	
model3-add	$\tilde{x}_i$	$m_i^q + \text{batch_shift_post}[b_i]$	$m_i^p + \text{batch_shift_prior}[b_i]$	$\tilde{x}_i$	
model4	$\tilde{x}_i$	m_i^q	m_i^p	$[z_i, \tilde{x}_i]$	
model5	$\tilde{x}_i, e_{b_i}$	m_i^q	m_i^p	$[z_i, \tilde{x}_i]$	
model6	$\tilde{x}_i, e_{b_i}$	m_i^q	m_i^p	$\tilde{x}_i$	

$[\cdot, \cdot]$ is `torch.cat` along the last axis. A comma inside `decoder_z_to_x(\cdot)` or `encoder_x_to_z(\cdot)` denotes `ConditionalDenseNN`'s `context` argument, which is concatenated onto the input at the *first* layer — not added to the output.

model1-add — Decipher-VZ-Add

(decipher.py L203–271)

$$z_i \sim \mathcal{N}(\text{encoder_x_to_z}(\tilde{x}_i), \cdot), \quad v_i \sim \mathcal{N}(\text{encoder_zx_to_v}([z_i, \tilde{x}_i]), \cdot)$$

$$\begin{aligned} \mathcal{L}_i = & \log \text{NegativeBinomial} \left(x_i \mid \text{softmax}(\text{decoder_z_to_x}(z_i)), \text{theta} \right) \\ & + \log \mathcal{N} \left(z_i \mid \text{decoder_v_to_z}(v_i) + \text{batch_shift}[b_i], \text{softplus}(\tilde{s}_i^p) \right) \\ & - \log \mathcal{N} \left(z_i \mid \text{encoder_x_to_z}(\tilde{x}_i), \text{softplus}(\tilde{s}_i^q) + \epsilon \right) \\ & + \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_zx_to_v}([z_i, \tilde{x}_i]), \cdot) \right] \end{aligned} \quad (6)$$

`guide()` accepts `batch_idx` and deliberately ignores it (L243): this arm's guide is $q(z \mid x)$. Note `batch_shift` occurs in *exactly one* term.

model2-add — Decipher-VZ2-Add

(L206–265)

$$\begin{aligned} \mathcal{L}_i = & \log \text{NegativeBinomial} \left(x_i \mid \text{softmax}(\text{decoder_z_to_x}(z_i)), \text{theta} \right) \\ & + \log \mathcal{N} \left(z_i \mid \text{decoder_v_to_z}(v_i) + \text{batch_shift_prior}[b_i], \cdot \right) \\ & - \log \mathcal{N} \left(z_i \mid \text{encoder_x_to_z}(\tilde{x}_i) + \text{batch_shift_post}[b_i], \cdot \right) \\ & + \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_zx_to_v}([z_i, \tilde{x}_i]), \cdot) \right] \end{aligned} \quad (7)$$

Two distinct tables since fix 2. When they were a single shared object, `batch_shift` cancelled exactly out of the z terms on every sample, deleting the gradient that identifies it.

model3-add — Decipher-MF-Add

(L209–269)

Identical to `model2-add` except the final line, where z_i never enters:

$$+ \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_x_to_v}(\tilde{x}_i), \cdot) \right] \quad (8)$$

model4 — Decipher-ZX

(L229–306)

$$\begin{aligned}
\mathcal{L}_i = & \log \text{NegativeBinomial} \left(x_i \mid \text{softmax}(\text{decoder_z_to_x}(z_i, e_{b_i})), \text{theta} \right) \\
& + \log \mathcal{N} \left(z_i \mid \text{decoder_v_to_z}(v_i), \cdot \right) \\
& - \log \mathcal{N} \left(z_i \mid \text{encoder_x_to_z}(\tilde{x}_i), \cdot \right) \\
& + \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_zx_to_v}([z_i, \tilde{x}_i]), \cdot) \right]
\end{aligned} \tag{9}$$

b_i appears *once*, inside the likelihood. Both z terms are batch-blind.

model5 — Decipher-ZX2

(L221–293)

$$\begin{aligned}
\mathcal{L}_i = & \log \text{NegativeBinomial} \left(x_i \mid \text{softmax}(\text{decoder_z_to_x}(z_i, e_{b_i})), \text{theta} \right) \\
& + \log \mathcal{N} \left(z_i \mid \text{decoder_v_to_z}(v_i), \cdot \right) \\
& - \log \mathcal{N} \left(z_i \mid \text{encoder_x_to_z}(\tilde{x}_i, e_{b_i}), \cdot \right) \\
& + \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_zx_to_v}([z_i, \tilde{x}_i]), \cdot) \right]
\end{aligned} \tag{10}$$

e_{b_i} enters `encoder_x_to_z` as a concatenated *input*, so it reaches both output heads (m_i^q and $\tilde{s}_i^q$ come from one final `Linear`, split afterwards) and its effect is a function of $\tilde{x}_i$ rather than a constant per batch.

model6 — Decipher-MF2

(L227–300)

Identical to `model5` except the final line:

$$+ \beta \left[\log \mathcal{N}(v_i \mid 0, I) - \log \mathcal{N}(v_i \mid \text{encoder_x_to_v}(\tilde{x}_i), \cdot) \right] \tag{11}$$

Two things visible directly in these expressions

model1-add and model2-add are the same model. Substituting `encoder_x_to_z`⁽²⁾ $(\tilde{x}_i) = \text{encoder_x_to_z}$ ⁽¹⁾ $(\tilde{x}_i) - \text{batch_shift_post}[b_i]$ makes every line coincide: the sampled z_i is the same, the z -term difference is the same, and $\log q(z_i)$ is the same. Both encoders are unconstrained networks of $\tilde{x}_i$ alone with identical capacity, so the two induce the same joint distribution over (v, z, x) .

Whether the reconstruction gradient can reach the batch parameter. In `model1-add-model3-add`, `batch_shift` (resp. `batch_shift_prior`, `batch_shift_post`) occurs only in the z terms. The z_i that reaches `decoder_z_to_x` is the guide’s draw, which contains no `batch_shift`, so

$$\frac{\partial}{\partial \text{batch_shift}} \log p(x_i \mid z_i) = 0 \quad \text{exactly, on every sample.} \tag{12}$$

In `model4-model6`, the batch parameters are the first B columns of `decoder_z_to_x`’s weight, i.e. they sit *inside* $\log p(x_i \mid \cdot)$ — the largest term in the objective. That single structural difference is what the empirical results turn on.